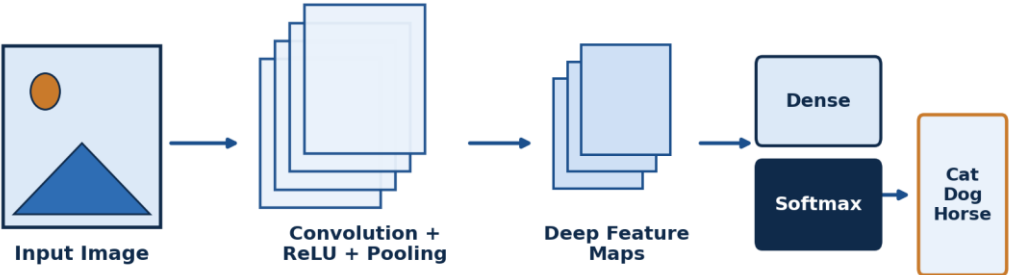


AUTOMATED IMAGE CLASSIFICATION USING CONVOLUTIONAL NEURAL NETWORKS

A Deep Learning-Based Case Study for Automated Image Classification

Input Image → Convolutional Neural Network → Feature Maps → Classification



Student Name	Paleru Dharani Govardhan
Register No.	192525280
Department	Artificial Intelligence and Machine Learning
Course Code	MLA04
Course	Deep Learning
CO Assessed	CO3
Assessment Tool	AT1 – Case Study Analysis
Total Marks	25

Abstract

Automated image classification is the task of assigning a predefined category label to a digital image using a computational model rather than manual human judgement. Colleges, businesses, and research institutions increasingly generate large volumes of image data that must be organised, tagged, and classified, and manual classification of such volumes is time-consuming, labour-intensive, inconsistent, and difficult to scale. This case study analyses the design of an automated image classification system built using a Convolutional Neural Network (CNN), a class of Deep Learning architecture that is specifically suited to processing grid-structured visual data such as images.

The report explains how a CNN automatically learns hierarchical visual features — edges, textures, shapes, and eventually complete objects — directly from raw pixel data, removing the need for manually engineered features such as those used in classical computer vision. The proposed pipeline covers data collection, image preprocessing, convolutional feature extraction, non-linear activation, spatial pooling, fully connected classification layers, and a softmax output layer that produces class probabilities. The report further discusses model training using backpropagation and the Adam optimizer, evaluation using standard classification metrics, systematic debugging strategies for issues such as overfitting and underfitting, and software engineering principles — modularity, scalability, maintainability, security, and reliability — that should govern the system's implementation.

The expected benefit of the proposed system is a faster, more consistent, and more scalable classification pipeline that reduces manual effort while supporting continuous improvement through retraining. The report also acknowledges practical challenges, including dependency on data quality and volume, computational cost, and the need for careful hyperparameter tuning, and it does not claim any specific numerical accuracy since no experimental training has been carried out as part of this analysis.

Introduction

Image classification is the process of analysing the visual content of an image and assigning it to one of a set of predefined categories. It is one of the foundational tasks in computer vision and underlies applications such as document sorting, product categorisation, medical image screening, quality inspection, and content organisation. In an academic or institutional setting, a college may accumulate large collections of images — scanned documents, photographs, diagrams, or object images — that need to be systematically classified for archival, retrieval, or administrative purposes.

Traditional, manual classification requires a human reviewer to examine each image individually and assign it to a category. While this approach can work adequately for very small collections, it becomes impractical as the number of images grows into the thousands or millions. Manual review is slow, expensive in terms of human effort, subject to fatigue-driven errors, and inconsistent between different reviewers or even across sessions performed by the same reviewer. These limitations create a strong motivation for automation.

Deep Learning offers a solution by allowing a model to learn the mapping between raw image pixels and category labels directly from labelled examples, without requiring a human expert to hand-design the rules or features used for classification. Among Deep Learning architectures, the Convolutional Neural Network (CNN) is particularly well suited to image data because it exploits the spatial structure of images — nearby pixels are related, and visual patterns such as edges and textures recur at different locations within an image. CNNs use convolutional filters that scan across the image to detect these local patterns, and they progressively combine simple patterns into more complex, higher-level representations of objects. This hierarchical feature learning is the central reason CNNs have become the standard architecture for image classification tasks.

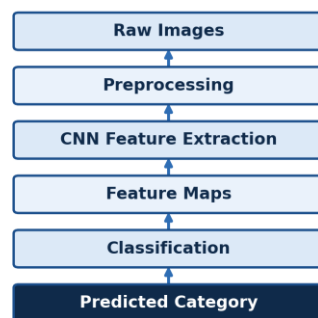


Figure 1: High-level pipeline from raw images to a predicted category.

1. Understand the Problem

1.1 Problem Explanation

In simple terms, the system described in this case study receives a digital image as input and must predict which predefined category the image belongs to. The CNN is trained beforehand on a labelled dataset so that it learns the relationship between visual patterns and category labels. Once trained, the model can be given a new, previously unseen image, and it will output a probability score for every possible category. The category with the highest probability score is selected as the model's prediction.

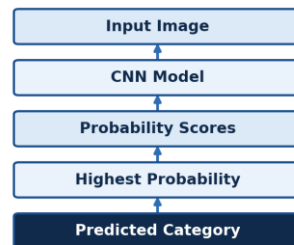


Figure 2: Conceptual flow from an input image to a predicted category.

The following is an illustrative example only, intended to demonstrate how the model's output should be interpreted. It does not represent real experimental results.

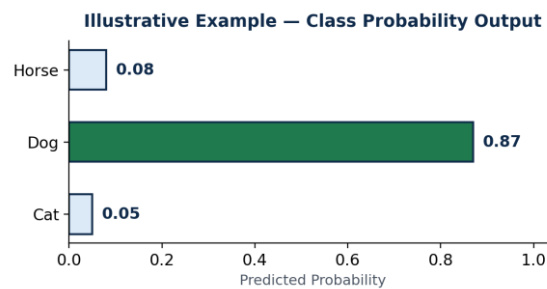


Figure 3 (Illustrative Example): For a given input image, the model outputs a probability for each class — here, Dog receives the highest probability (0.87) and is selected as the prediction.

1.2 Objectives

1. Automate the classification of images into predefined categories without manual intervention.
2. Reduce the manual effort currently required to sort and label large image collections.
3. Improve consistency of classification decisions across the entire dataset.
4. Enable the model to automatically learn relevant visual features rather than relying on hand-crafted rules.
5. Support classification of large-scale image datasets in a scalable manner.
6. Provide fast predictions suitable for near real-time or batch use cases.
7. Build a classification system whose architecture can scale with growing data volume.
8. Evaluate model performance using standard, well-understood classification metrics.
9. Create a maintainable software architecture that supports future updates.
10. Support future improvements to the model as more data and feedback become available.

1.3 Scope

In Scope

- Collection of a labelled image dataset covering the target categories.
- Image preprocessing, including resizing, normalisation, and augmentation.
- Preparation of training, validation, and test dataset splits.
- Design and training of a Convolutional Neural Network.
- Automatic feature extraction through convolutional layers.
- Multi-class classification of input images.
- Evaluation of model performance using appropriate metrics.

- Generation of predictions for new, unseen images.
- Debugging and refinement of the trained model.
- Deployment of the trained model as part of a maintainable software system.

Out of Scope

- Manual image labelling activities beyond the initial dataset preparation stage.
- Human-level interpretation of every conceivable edge-case image.
- Processing of unrelated, non-image data types such as text or audio.
- Any guarantee of 100% classification accuracy under all conditions.

1.4 Type of Deep Learning Problem

This case study addresses a **Supervised Deep Learning** problem, and more specifically a **Multi-Class Image Classification** problem.

- Input: a digital image, typically represented as a Height × Width × Channels tensor.
- Target: a class label drawn from a fixed, predefined set of categories.
- Training data: a set of labelled images, where each image is paired with its correct category.
- Output: a predicted class for each new input image, derived from the class with the highest predicted probability.

If the classification task involves only two categories (for example, defective vs. non-defective), the same architecture reduces to a binary classification problem, typically using a sigmoid output rather than softmax. The design described in this report generalises to both cases, with the multi-class formulation as the primary focus.

1.5 Why Deep Learning Is Suitable

- Automatic feature extraction: CNNs learn relevant visual features directly from data, removing the need for manual feature engineering.
- Spatial feature learning: convolutional filters exploit the two-dimensional spatial structure of images.
- Hierarchical representation: early layers learn simple features (edges, colours) while deeper layers learn complex features (shapes, objects).
- High-dimensional data handling: CNNs can efficiently process the large number of pixel values present in typical images.
- Reduced manual effort: feature design work that classical computer vision required is largely automated.
- Strong classification capability: CNNs consistently perform well on image classification benchmarks.
- Scalability: CNN-based pipelines can be extended to larger datasets and additional categories with comparatively modest architectural changes.

2. Data and Challenges

2.1 Stakeholders

A number of stakeholders are involved in the successful development and operation of the automated image classification system. Each stakeholder has a distinct role, and their responsibilities are summarised in Table 1.

Stakeholder	Responsibility
College / Organization	Defines classification requirements, categories, and success criteria for the system.
End User	Uses the classification system to upload images and view predicted categories.
Data Engineer	Collects, cleans, and organises image data into a usable dataset.
ML Engineer	Designs, trains, tunes, and evaluates the CNN model.
Software Developer	Builds the surrounding application, including APIs and user interface.
System Administrator	Manages deployment infrastructure, servers, and system uptime.
Domain Expert	Validates that image labels and categories are correct and meaningful.

Stakeholder	Responsibility
Project Manager	Coordinates the development schedule, resources, and deliverables.

2.2 Dataset

A suitable dataset for this case study consists of a large collection of images, each associated with a correct class label. The dataset should be organised into separate directories for training, validation, and testing, with each directory further divided into sub-folders representing each class.

```
dataset/
├── train/
│   ├── class_A/
│   ├── class_B/
│   └── class_C/
├── validation/
│   ├── class_A/
│   ├── class_B/
│   └── class_C/
└── test/
    ├── class_A/
    ├── class_B/
    └── class_C/
```

- Training set: used to update the model's weights during learning.
- Validation set: used during development to tune hyperparameters and monitor for overfitting, without directly influencing weight updates.
- Test set: held out entirely until final evaluation, providing an unbiased estimate of real-world performance.

2.3 Input and Output

Input

Each input image is represented as a three-dimensional tensor:

Height × Width × Channels
Example: 224 × 224 × 3

- 224 — image height in pixels.
- 224 — image width in pixels.
- 3 — number of colour channels (Red, Green, Blue).

Output

The model produces a probability distribution over all candidate classes:

Class A → 0.10
Class B → 0.75
Class C → 0.15

Prediction → Class B

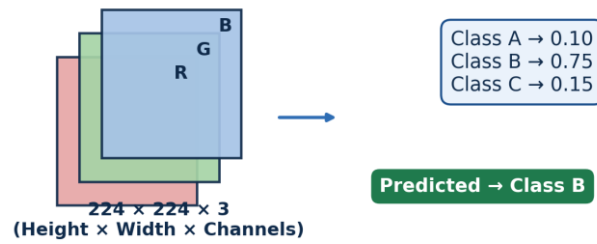


Figure 4: Representation of an RGB image tensor and its corresponding class probability output.

2.4 Data Challenges

- Insufficient training data for one or more categories.
- Class imbalance, where some categories have far more examples than others.
- Poor image quality, including blur, compression artefacts, and low resolution.
- Variation in lighting conditions across images.
- Variation in backgrounds behind the subject of interest.
- Rotation of the subject within the frame.
- Scale variation, where the same object appears at different sizes.
- Occlusion, where part of the object of interest is hidden.
- Duplicate images that can bias training or evaluation.
- Incorrectly labelled images introduced during dataset preparation.
- Background noise or irrelevant visual artefacts in the image.

2.5 Technical Challenges

- High computational requirements for training deep networks.
- Dependency on GPU acceleration for practical training times.
- Long training time for large datasets or deep architectures.
- High memory usage during training, particularly with large batch sizes.
- Model size, which affects storage and deployment footprint.
- Inference latency, which affects real-time usability.
- Risk of overfitting to the training data.
- Ensuring generalisation to unseen, real-world images.
- Complexity of deploying and serving a trained model reliably.
- Scalability of the system as image volume and user load increase.

Challenge	Impact	Solution
Insufficient data	Poor generalisation, overfitting	Data augmentation, transfer learning, more data collection
Class imbalance	Bias toward majority class	Class weighting, oversampling, targeted collection
High computation cost	Slow training, high infrastructure cost	GPU acceleration, efficient architectures, cloud resources
Overfitting	High train accuracy, low validation accuracy	Dropout, regularisation, early stopping, augmentation
Deployment complexity	Difficult integration and scaling	Modular architecture, containerisation, API-based design

Image Preprocessing

Before an image can be used by a CNN, it must pass through a structured preprocessing pipeline. This pipeline standardises image format and size, scales pixel values into a numerically stable range, and optionally expands the effective size of the training set through data augmentation.

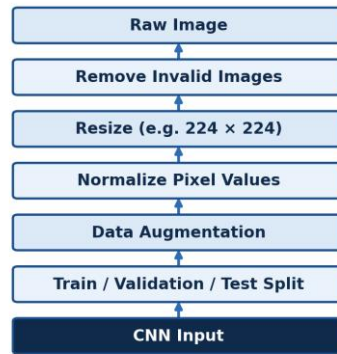


Figure 5: Complete image preprocessing pipeline from raw image to CNN input.

Resizing

Images collected from different sources typically vary in resolution and aspect ratio. CNNs require a fixed input size, so every image is resized to a common dimension before it is passed to the network. A typical choice for image classification tasks is:

224 × 224

Resizing ensures that all images share the same tensor shape, which is required because the network's convolutional and fully connected layers are defined for a fixed input size.

Normalization

Raw pixel values typically range from 0 to 255 for each colour channel. Normalisation rescales these values into a smaller, numerically stable range, which improves training stability and convergence speed. A common normalisation approach divides each pixel value by 255:

Pixel / 255

This operation maps the original range of approximately 0–255 to approximately 0–1, which is a more suitable input range for gradient-based optimisation.

Data Augmentation

Data augmentation artificially increases the diversity of the training set by applying label-preserving transformations to existing images. This helps the model generalise better and reduces the risk of overfitting, particularly when the available dataset is limited in size. Common augmentation techniques include:

- Rotation — rotating the image by a small random angle.
- Horizontal flip — mirroring the image left-to-right.
- Vertical flip — used only where a vertical mirror still represents a valid example of the class.
- Zoom — randomly zooming in or out of the image.
- Crop — extracting a random sub-region of the image.
- Translation — shifting the image horizontally or vertically.
- Brightness variation — randomly adjusting image brightness.

It is important that every augmentation preserves the correct class meaning of the image. For example, a vertical flip may be inappropriate for a category where orientation is meaningful (such as distinguishing a correctly oriented object from an inverted one), and augmentation strategies must be chosen with this constraint in mind.

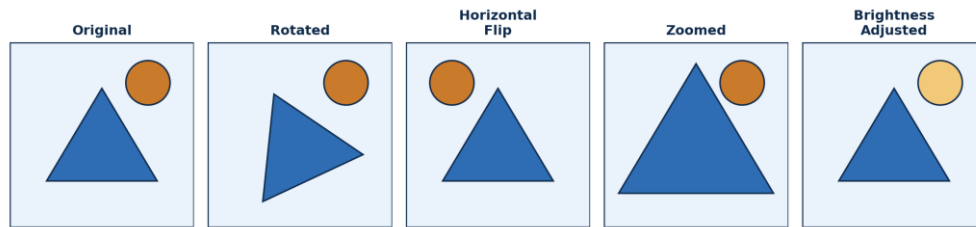


Figure 6: Example data augmentation transformations applied to a single training sample.

3. Existing Approach

Before evaluating a CNN-based solution, it is important to analyse how image classification has traditionally been performed, and to understand the specific gap that Deep Learning addresses.

Manual Classification

In the manual approach, a human reviewer visually inspects each image and assigns it to what they judge to be the correct category. The result is then recorded, typically in a spreadsheet, database, or filing system.

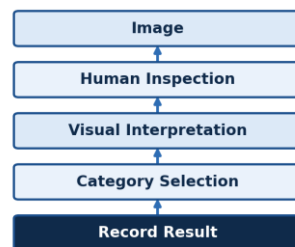


Figure 7: Workflow of traditional manual image classification.

3.1 Strengths

- Human reviewers bring contextual understanding that can resolve ambiguous cases.
- Humans can handle unusual or previously unseen cases using general reasoning.
- Manual review is straightforward and effective for very small datasets.
- No model training, infrastructure, or technical setup is required.

3.2 Limitations

- Manual classification is slow relative to the volume of images that must be processed.
- It becomes expensive at scale due to the human labour required.
- Reviewer fatigue leads to increased error rates over long sessions.
- Different reviewers (or the same reviewer at different times) may reach inconsistent decisions.
- Judgements can be subjective, particularly for borderline cases.
- It is practically infeasible to manually review datasets containing millions of images.
- The approach offers limited opportunities for automation or integration into larger systems.

3.3 Traditional Computer Vision Alternative

Prior to the widespread adoption of Deep Learning, classical computer vision techniques attempted to automate image classification using hand-engineered feature extraction methods, combined with conventional machine learning classifiers. Representative techniques include:

- Edge detection (for example, Canny or Sobel operators) to capture object boundaries.
- Scale-Invariant Feature Transform (SIFT) for detecting distinctive local keypoints.
- Histogram of Oriented Gradients (HOG) for capturing local shape information.
- Colour histograms for representing overall colour distribution.
- Texture descriptors for capturing surface pattern information.
- Classification using algorithms such as Support Vector Machines (SVM) or Random Forests, applied to the extracted features.

The central limitation of this approach is that the features themselves must be manually designed by an expert. This manual feature engineering step is time-consuming, requires domain expertise, and often fails to generalise well across different datasets or visual conditions.

3.4 Gap Addressed by CNN

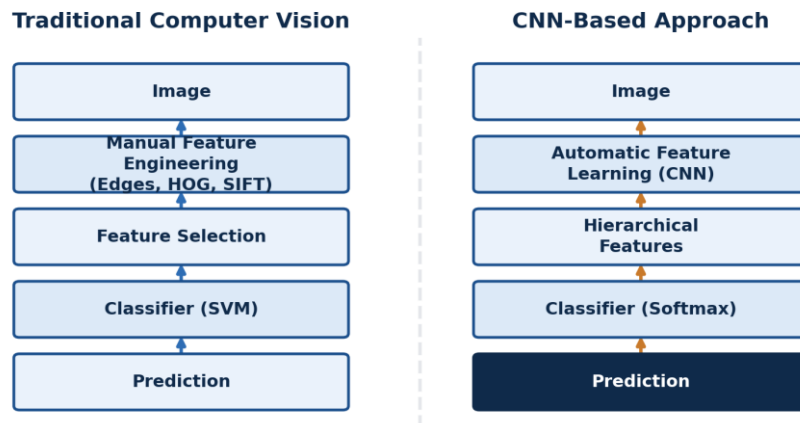


Figure 8: Comparison between the traditional computer vision pipeline (manual feature engineering) and the CNN-based pipeline (automatic feature learning).

A CNN replaces the manual feature engineering stage with a learned feature extraction process. Instead of an expert deciding which edges, textures, or shapes are relevant, the network learns the most useful features directly from labelled training data through gradient-based optimisation. This reduces dependence on manually designed visual features, allows the model to discover patterns that may not be obvious to a human designer, and generally produces stronger and more transferable performance across a wide range of image classification tasks.

4. Proposed CNN Solution

This case study proposes a Convolutional Neural Network as the core model for the automated image classification system. The general architecture proceeds through repeated blocks of convolution, non-linear activation, and pooling, followed by a classification head that outputs class probabilities.



Figure 9: Conceptual CNN processing flow from input image to class prediction.

4.1 CNN Components

Input Layer

Receives the preprocessed image tensor, typically shaped as Height × Width × Channels.

Convolution Layer

Applies a set of learnable filters (kernels) across the image to extract local visual features such as edges, corners, and textures.

ReLU (Rectified Linear Unit)

Introduces non-linearity into the network, allowing it to model complex, non-linear relationships between pixels and class labels.

Pooling

Reduces the spatial dimensions of feature maps, decreasing computational cost while retaining the most important information.

Feature Maps

The outputs of convolutional layers, representing the presence and location of learned visual patterns at each layer.

Fully Connected / Classification Layer

Maps the extracted, flattened features to the set of target classes.

Softmax

Converts the final layer's raw outputs into a probability distribution across all candidate classes, suitable for multi-class classification.

4.2 Convolution Mathematics

A convolution operation can be represented conceptually as:

$$\text{Feature Map} = \text{Input} \times \text{Kernel} + \text{Bias}$$

More formally, for a two-dimensional convolution, each element of the output feature map is computed as the sum of element-wise products between a small region of the input and the kernel, plus a bias term:

$$\text{Output}(i, j) = \sum \sum \text{Input}(i+m, j+n) \cdot \text{Kernel}(m, n) + b$$

- Kernel: a small matrix of learnable weights (for example, 3×3) that slides across the input.
- Stride: the number of pixels the kernel moves at each step.
- Padding: additional border pixels (often zeros) added to control the output spatial size.
- Feature map: the resulting output produced by sliding the kernel across the entire input.

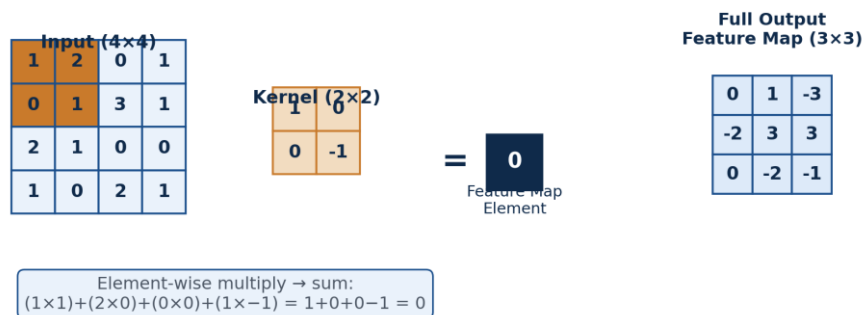


Figure 10: Simple numerical example of a 2×2 convolution kernel applied to a 4×4 input, producing one element of a 3×3 output feature map.

4.3 ReLU

$$\text{ReLU}(x) = \max(0, x)$$

ReLU is the most commonly used activation function in CNN hidden layers because it is computationally efficient, helps mitigate the vanishing gradient problem for positive activations, and empirically leads to faster and more stable convergence compared to older activation functions such as sigmoid or tanh.

4.4 Pooling

Max Pooling

Selects the maximum value from each pooling region, preserving the strongest activation within that region.

Average Pooling

Computes the average value across each pooling region, producing a smoother, less peaked summary.

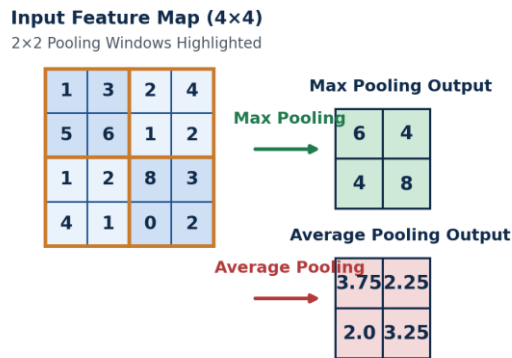


Figure 11: Comparison of max pooling and average pooling applied to the same 4×4 feature map.

Pooling reduces spatial dimensions and computation in subsequent layers, and provides a degree of translation tolerance — meaning the network becomes less sensitive to small shifts in the position of a detected feature within the image.

4.5 Softmax

For multi-class classification, the softmax function converts the final layer's raw output scores into a probability distribution across all classes:

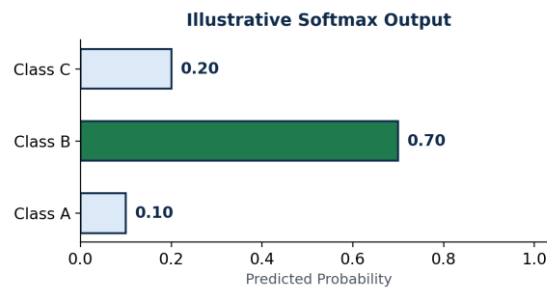


Figure 12 (Illustrative): Softmax converts raw scores into normalised class probabilities that sum to approximately 1.

The resulting probabilities always sum to approximately 1, and the class with the highest probability is taken as the model's prediction.

CNN Architecture and Training

The following is a proposed, illustrative CNN architecture for the automated image classification task. It is not the result of experimental tuning, but represents a reasonable, commonly used baseline structure for a small-to-moderate image classification problem.

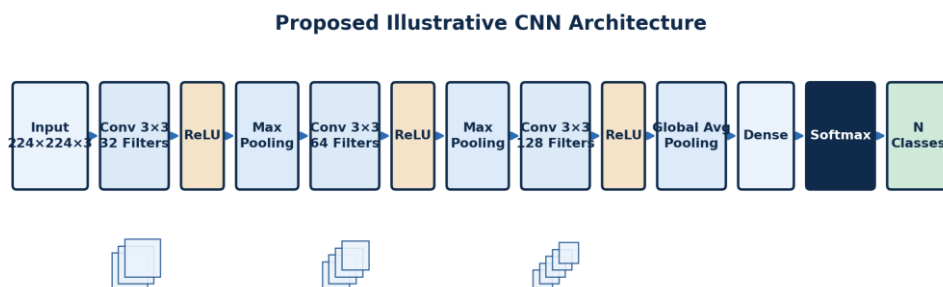


Figure 13 (Proposed Illustrative Architecture): Input → three convolution/ReLU/pooling blocks with increasing filter depth (32, 64, 128) → global average pooling → dense layer → softmax → N classes.

4.6 Training Process

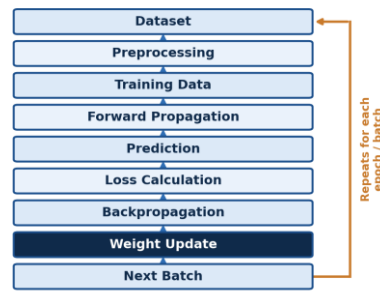


Figure 14: Iterative training loop showing forward propagation, loss calculation, backpropagation, and weight updates.

During training, the dataset is first preprocessed and organised into batches. For each batch, the network performs forward propagation to generate predictions, and a loss function quantifies how far these predictions are from the true labels. Backpropagation then computes the gradient of the loss with respect to each weight in the network, and an optimizer uses these gradients to update the weights in a direction that reduces the loss. This process repeats across all batches in the dataset.

- Epoch: one complete pass through the entire training dataset.
- Batch: a subset of the training data processed together in a single forward/backward pass; batching makes training computationally feasible for large datasets.

Training typically continues for multiple epochs until the model's performance on the validation set stops improving, at which point training may be stopped to avoid overfitting.

4.7 Transfer Learning

Where the available labelled dataset is limited in size, transfer learning offers a practical alternative to training a CNN from scratch. A model that has already been pretrained on a large, general-purpose image dataset can be adapted to the college's specific classification task by reusing its learned feature extraction layers and retraining only the final classification layers.

- ResNet — a deep architecture using residual connections to ease training of very deep networks.
- VGG — a straightforward, deep architecture using small, stacked convolutional filters.
- DenseNet — an architecture with dense connections between layers, encouraging feature reuse.
- EfficientNet — a family of architectures designed to balance accuracy and computational efficiency.
- MobileNet — a lightweight architecture optimised for mobile and resource-constrained deployment.



Figure 15: Transfer learning workflow — reuse a pretrained CNN, replace its classification head, and fine-tune on the target dataset.

Transfer learning is particularly useful when the college has a limited number of labelled images, since the pretrained model has already learned general-purpose visual features (edges, textures, shapes) from a much larger dataset, and only needs to adapt these features to the specific target categories.

Loss Function, Optimizer, and Performance Metrics

Loss Function — Categorical Cross-Entropy

For multi-class classification, the recommended loss function is Categorical Cross-Entropy. Conceptually, this loss measures how different the model's predicted probability distribution over classes is from the true class distribution (where the correct class has probability 1 and all other classes have probability 0). A lower cross-entropy loss indicates that the model's predicted probabilities are closer to the correct answer.

Where class labels are provided as integers rather than one-hot encoded vectors, Sparse Categorical Cross-Entropy performs the same underlying computation without requiring the labels to be pre-converted into one-hot form.

Optimizer — Adam

The Adam optimizer is recommended for training the proposed CNN. Adam combines the benefits of adaptive learning rates with momentum-based updates, making it well suited to a wide range of Deep Learning tasks.

- Adaptive learning rate: Adam adjusts the effective learning rate for each parameter individually, based on estimates of recent gradient magnitudes.
- Efficient optimization: Adam generally converges faster than plain stochastic gradient descent on many practical tasks.
- Practical training: Adam requires comparatively little manual tuning to achieve reasonable performance.
- Common use in CNN training: Adam is one of the most widely used optimizers for image classification tasks in current practice.

Regardless of the recommended default, the specific choice of optimizer and its learning rate must be validated experimentally on the actual dataset, since optimal settings can vary between tasks.

Performance Metrics

Accuracy

The proportion of predictions that exactly match the true label, across all classes.

Precision

Of all instances the model predicted as belonging to a given class, the proportion that were actually correct.

Recall

Of all instances that actually belong to a given class, the proportion that the model correctly identified.

F1-Score

The harmonic mean of precision and recall, providing a single balanced measure when there is a trade-off between the two.

Confusion Matrix

A table showing the relationship between predicted and actual classes, structured around four fundamental outcomes for binary classification:

- True Positive (TP): correctly predicted positive case.
- True Negative (TN): correctly predicted negative case.
- False Positive (FP): incorrectly predicted as positive.
- False Negative (FN): incorrectly predicted as negative.

For multi-class classification, the confusion matrix is extended to contain one row and one column for each class, where the diagonal entries represent correct predictions and off-diagonal entries represent specific types of misclassification between class pairs.

Model Evaluation

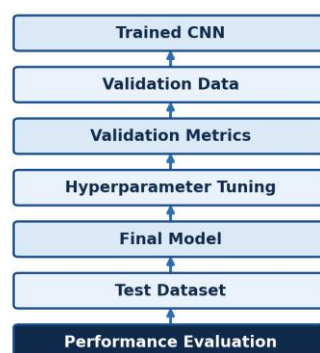


Figure 16: Evaluation workflow from a trained CNN through hyperparameter tuning to final test-set performance evaluation.

Model evaluation proceeds in stages. During development, the model's performance is repeatedly checked against the validation set, and hyperparameters are adjusted based on these results. Once a final configuration is selected, the model is evaluated exactly once on the held-out test set to obtain an unbiased estimate of real-world performance. The following quantities are typically tracked throughout this process:

- Training accuracy — accuracy measured on the data the model was trained on.
- Validation accuracy — accuracy measured on data not used for weight updates, used to monitor generalisation.
- Training loss — the loss value computed on training data.
- Validation loss — the loss value computed on validation data.

- Confusion matrix — a detailed breakdown of correct and incorrect predictions by class.
- Precision, Recall, and F1-score — computed per class and, where useful, averaged across classes.

The following training curves are illustrative examples only, intended to demonstrate the typical shape of accuracy and loss curves over training. They do not represent actual experimental results, since no model has been trained as part of this case study.

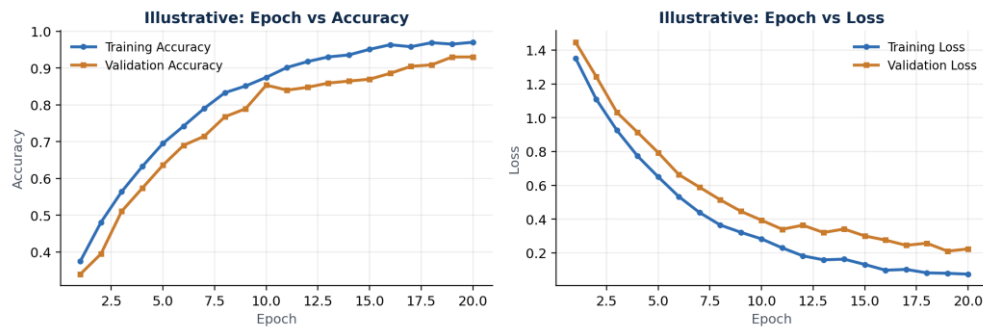


Figure 17 (Illustrative): Epoch vs. training/validation accuracy (left) and epoch vs. training/validation loss (right).

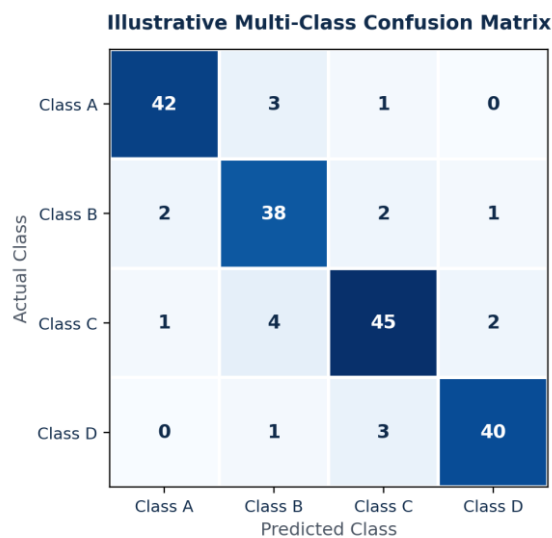


Figure 18 (Illustrative): Multi-class confusion matrix showing predicted vs. actual class counts.

5. Model and Methodology Justification

Why CNN?

Approach	Suitability for Image Classification
Traditional Image Processing	Requires manual feature engineering; does not scale well to complex or varied visual data.
Fully Connected Neural Network	Ignores spatial structure of images; requires an impractically large number of parameters for realistic image sizes.
Recurrent Neural Network (RNN)	Designed for sequential data such as text or time series; not naturally suited to two-dimensional spatial image data.
LSTM	Designed to capture long-range dependencies in sequences; offers no particular advantage for static image classification.
Convolutional Neural Network (CNN)	Exploits spatial locality and translation invariance in images; automatically learns hierarchical visual features; the standard and most effective architecture for image classification.

CNNs are the appropriate primary architecture for this case study because the input data is inherently spatial — the arrangement of pixels relative to one another carries meaning. Convolutional filters are specifically designed to exploit this local spatial structure, and the parameter-sharing property of convolution (the same filter is applied across the whole image) makes CNNs far more parameter-efficient than a fully connected network operating directly on raw pixels.

Activation Function Justification

ReLU

Used in hidden convolutional layers because it introduces non-linearity, is computationally efficient to compute and differentiate, and provides better gradient flow for positive activations compared with saturating activation functions such as sigmoid or tanh.

Softmax

Used in the final output layer for multi-class classification because it produces a normalised probability distribution across all classes, which is appropriate when the target categories are mutually exclusive (each image belongs to exactly one class).

Loss Function Justification

Categorical Cross-Entropy is used because it directly measures the discrepancy between the predicted class probability distribution and the true one-hot encoded label, and its gradient properties work well in combination with a softmax output layer during backpropagation, making it the standard choice for multi-class classification problems of this kind.

Optimizer Justification

Adam is recommended because it adapts the learning rate for each parameter individually, incorporates momentum to speed convergence, and generally requires less manual tuning than plain stochastic gradient descent, making it a practical default starting point for training the proposed CNN.

Hyperparameter Tuning

A number of hyperparameters influence CNN training and performance and must be tuned appropriately for the specific dataset and task:

- Learning rate — controls the size of weight updates during optimisation.
- Batch size — the number of training examples processed together in one forward/backward pass.
- Number of epochs — how many times the model passes over the full training dataset.
- Number of filters — the number of feature detectors learned in each convolutional layer.
- Kernel size — the spatial dimensions of each convolutional filter.
- Dropout rate — the proportion of neurons randomly deactivated during training to reduce overfitting.
- Weight decay — a regularisation term that penalises large weight values.
- Data augmentation strength — how aggressively augmentation transformations are applied.
- Optimizer choice — the specific algorithm used to update model weights.

Common tuning strategies include grid search (exhaustively testing combinations of predefined values), random search (sampling combinations randomly, often more efficient than grid search for high-dimensional hyperparameter spaces), and Bayesian optimisation (using a probabilistic model to intelligently select promising combinations to evaluate).

The following table lists illustrative search values only, intended to demonstrate typical ranges considered during tuning; they are not the result of an actual tuning experiment.

Hyperparameter	Illustrative Search Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Kernel Size	3×3, 5×5
Epochs	20, 40, 60
Dropout	0.2, 0.5

Hyperparameter	Illustrative Search Values
Optimizer	Adam, SGD

Software Engineering Principles

Beyond the CNN model itself, the overall system must be engineered according to sound software engineering principles to remain usable, extensible, and trustworthy over time.

Modularity

The system should be organised into clearly separated components, each responsible for a distinct concern:

- Data processing (loading, preprocessing, augmentation)
- Model definition and training
- Evaluation and metrics computation
- Prediction / inference
- API layer for external communication
- User interface for interaction with end users

Scalability

The system should be designed to support growth in:

- Volume of images processed
- Number of concurrent users
- Number of classification categories
- Volume of prediction requests over time

Maintainability

Long-term maintainability is supported through:

- Clean, well-organised, and well-documented code
- Externalised configuration files rather than hard-coded parameters
- Version control for all source code and configuration
- Clearly separated, modular components
- Comprehensive documentation of design decisions and interfaces

Security

The system must protect:

- User information collected through the application
- Uploaded images, which may be sensitive
- Trained model files, which represent a valuable asset
- API endpoints, which must be protected from misuse

Recommended security measures include authentication, authorization, input validation, and secure storage of both data and model artefacts.

Reliability

System reliability should be supported through:

- Robust error handling throughout the pipeline
- Comprehensive logging of system events and predictions
- Ongoing monitoring of system health and model performance
- Regular backups of data and trained models
- Model versioning to allow safe rollback if a new model underperforms

SDLC / Development Methodology

Given the iterative, experimental nature of Deep Learning development, this case study recommends the **Agile Software Development Methodology** as the appropriate development framework.

Deep Learning development is inherently iterative for several reasons:

- Data quality and characteristics can change as more images are collected.
- Model performance must be re-assessed after each change to data, architecture, or hyperparameters.
- Hyperparameters require ongoing experimentation rather than a single fixed configuration.
- Requirements from stakeholders may evolve as the project progresses.
- New categories may need to be added to the classification system over time.
- Model refinement is a continuous activity rather than a one-time task.

Agile development, with its short iterative cycles and continuous feedback, is well matched to this reality, allowing the team to adapt the model and system design incrementally rather than committing to a rigid upfront plan.

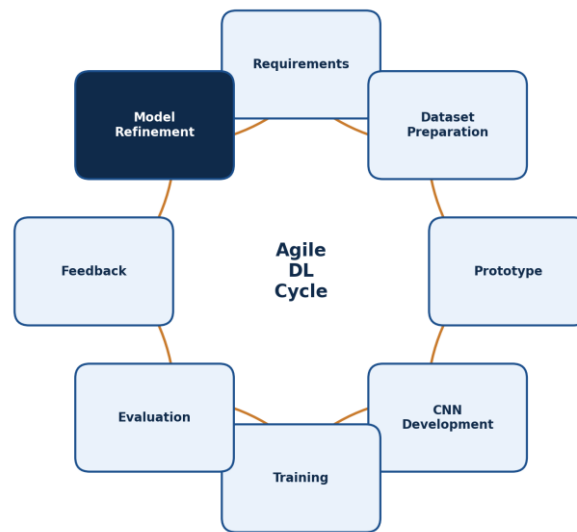


Figure 19: Agile development cycle adapted for iterative Deep Learning development.

MLOps Connection

As the project matures beyond an initial prototype, it can evolve into a full MLOps (Machine Learning Operations) workflow, which extends standard DevOps practices to address the specific needs of managing data, models, and continuous retraining.

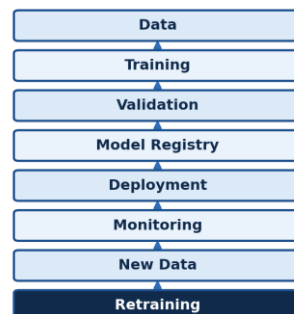


Figure 20: MLOps workflow — from data collection through training, deployment, monitoring, and retraining.

- Model versioning — tracking distinct trained versions of the model over time.
- Dataset versioning — tracking changes to the underlying training data.
- Experiment tracking — recording hyperparameters, metrics, and results for each training run.
- Deployment — releasing a validated model into the production environment.
- Monitoring — continuously tracking model performance and system health in production.

- Retraining — periodically updating the model as new labelled data becomes available.

Debugging and Model Refinement

CO3 explicitly requires the use of debugging strategies for model refinement. This section outlines common CNN performance problems, their likely causes, and corresponding solutions.

Problem: Training Accuracy Low

Possible Causes

- Model capacity is too small for the complexity of the task.
- Preprocessing is flawed or inconsistent.
- The learning rate is set too low, slowing convergence.
- The model has not been trained for a sufficient number of epochs.

Solutions

- Increase model capacity (more filters, additional layers).
- Review and improve the preprocessing pipeline.
- Tune the learning rate, potentially using a learning rate schedule.
- Train for additional epochs, monitoring for continued improvement.

Problem: Training Accuracy High, Validation Accuracy Low

This pattern indicates **Overfitting** — the model has effectively memorised the training data rather than learning generalisable patterns.

Possible Solutions

- Apply data augmentation to increase effective training data diversity.
- Introduce dropout layers to reduce co-adaptation of neurons.
- Apply L2 regularisation to penalise excessively large weights.
- Use early stopping to halt training when validation performance stops improving.
- Use transfer learning to leverage pretrained, more generalisable features.
- Collect more training data where feasible.

Problem: Both Training and Validation Accuracy Low

This pattern indicates **Underfitting** — the model is too simple, or has not learned sufficiently, to capture the underlying patterns in the data.

Possible Solutions

- Increase model capacity to allow the network to represent more complex patterns.
- Train for a longer duration.
- Improve the quality and relevance of input features and preprocessing.
- Reduce excessive regularisation that may be overly constraining the model.

Problem: Validation Performance Unstable

If validation performance fluctuates significantly between epochs, the following should be checked:

- Learning rate — an excessively high learning rate can cause unstable updates.
- Dataset quality — noisy or mislabelled data can produce inconsistent gradients.
- Batch size — very small batch sizes can introduce high variance into gradient estimates.
- Class imbalance — uneven class distribution can distort validation metrics between batches.
- Data leakage — inadvertent overlap between training and validation data can produce misleading, unstable results.

Debugging Workflow

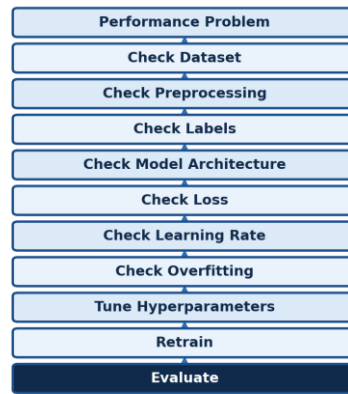


Figure 21: Systematic debugging workflow for diagnosing and resolving CNN performance issues.

6. Expected Outcomes, Limitations and Future Enhancements

Expected Outcomes

- Automated classification of images without requiring manual review of every item.
- Substantially reduced manual effort compared with fully manual classification.
- Faster processing of large image collections.
- More consistent classification decisions across the entire dataset.
- A scalable classification pipeline that can accommodate growing data volumes.
- Automatic extraction of relevant visual features without manual engineering.
- Improved overall productivity for stakeholders responsible for image organisation.

No specific accuracy percentages are claimed in this report, since no experimental training has been conducted as part of this case study; actual performance would need to be established through real training and evaluation on the target dataset.

Limitations

Dataset Dependency

The performance of the proposed CNN is fundamentally dependent on the quality, quantity, and diversity of the labelled training data. A model trained on a small, narrow, or biased dataset is unlikely to generalise well to the full range of images it will encounter in real-world use.

Computational Requirements

Training and, to a lesser extent, deploying a CNN requires meaningful computational resources, particularly GPU acceleration for practical training times. This has cost and infrastructure implications for the institution.

Generalisation Limits

Even a well-trained CNN may struggle with categories, conditions, or edge cases that were not adequately represented in the training data, such as unusual lighting, occlusion, or entirely novel visual categories.

Interpretability

CNNs are comparatively difficult to interpret compared with simpler models; understanding exactly why a particular prediction was made can require additional tools and techniques.

Maintenance Burden

A deployed classification system requires ongoing monitoring, periodic retraining, and infrastructure maintenance to remain accurate and reliable over time.

Future Enhancements

- Expanding the training dataset with additional labelled images across all categories.
- Adopting transfer learning with modern pretrained architectures to improve performance with limited data.
- Introducing more advanced data augmentation techniques to further improve generalisation.
- Building a full MLOps pipeline for continuous monitoring, versioning, and retraining.
- Adding explainability tools to help stakeholders understand model predictions.

- Extending the system to support additional classification categories as institutional needs evolve.
- Optimising the trained model for faster inference and lighter-weight deployment where required.

Conclusion

This case study has analysed the design of an automated image classification system built around a Convolutional Neural Network, addressing the core limitations of manual image classification through automatic, hierarchical feature learning. The report has covered the problem definition, stakeholders and dataset requirements, data and technical challenges, a comparison with traditional approaches, a detailed proposed CNN architecture with supporting mathematical explanations, training and evaluation methodology, justification of key design choices, software engineering principles, an appropriate Agile development methodology with an MLOps growth path, and systematic debugging strategies for model refinement. While actual performance figures would need to be established through real experimentation on a genuine dataset, the analysis presented here provides a technically grounded and practically actionable foundation for building such a system.